

# Lab 3

## Classes & Objects

Implementation, dry runs, debugging, complete solutions, and viva preparation for the [dynArr](#) class.

Dark Developer Handbook

Header + Source Files

Constructors + Destructors

Dynamic 1D Array

allocate(int s)

Dynamic 2D Array

Generated as a study-grade lab manual from the uploaded Lab 3 sheet. No unreadable task content was detected in the 3-page source PDF.

## Section 1 · Lab Map

This lab is about building a small class that owns dynamic memory. The innocent-looking class `dynArr` is really a training arena for object lifetime, heap allocation, file separation, getter/setter methods, resizing, and two-dimensional memory.

| Lab Target                   | What You Are Practicing                                                                             | Why It Matters                                                                       |
|------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Original <code>dynArr</code> | Class with private pointer, constructors, destructor, <code>setValue</code> , <code>getValue</code> | You learn to wrap unsafe heap memory inside an object.                               |
| Task 1                       | Create objects, input five values, print them                                                       | You prove that a class can store and return values through public methods.           |
| Task 2                       | Add <code>allocate(int s)</code>                                                                    | You learn safe resizing without leaking the old heap array.                          |
| Task 3                       | Extend to 2D equal-column array                                                                     | You learn row pointers, nested allocation, nested loops, and correct deletion order. |

**Skill built by the lab:** you should be able to design a class that owns memory, split it across `.h`, `.cpp`, and `main.cpp`, then explain exactly what happens when the object is constructed, used, resized, and destroyed.

## Section 2 - Page Extraction Map

The uploaded lab file has 3 pages. The map below preserves the source order and identifies every visible task, code block, and requirement.

### LAB 3

CSE225L



#### Classes & Objects

In this lab, we will:

- Implement and test the `dynarr` class using the header (.h) and source (.cpp) files.
- Demonstrate the use of constructors and destructors for creating and deleting arrays.
- Utilize the `setValue` and `getValue` methods to modify and access array elements.
- Modify the class to include functionality for dynamically resizing the array using `allocate(int s)`.
- Extend the class to support a 2D array where all rows have the same column size.

#### DYNARR

```
#ifndef DYNARR_H
#define DYNARR_H

class dynarr
{
private:
    int *data;
    int size;

public:
    dynarr();
    dynarr(int s);
    ~dynarr();
    void setValue(int index, int value);
    int getValue(int index);
};

#endif // DYNARR_H
```

```
#include "dynarr.h"
#include <iostream>
using namespace std;

dynarr::dynarr()
{
    data = NULL;
    size = 0;
}

dynarr::dynarr(int s)
{
    data = new int[s];
    size = s;
}

dynarr::~dynarr()
{
    delete[] data;
}

int dynarr::getValue(int index)
{
    return data[index];
}

void dynarr::setValue(int index, int value)
{
    data[index] = value;
}
```

#### DYNARR

#### TASKS:

**Task 1:** In the driver file (main.cpp), perform the following sub-tasks:

- Create two objects of this class: one with no constructor argument and one with the argument 5.
- Take five input values from the user and store them in the array inside the second object using the `setValue` method.
- For the second object, print all the values you just stored.

**Task 2:**

- Modify the header and source files to add a member function `allocate(int s)` that allows you to change the size of the array. Ensure that memory is not leaked.

**Task 3:**

- Modify the header and source files again to support a two-dimensional array where all rows are the same size. The user will specify both the number of rows and columns, as well as the content of the array. You will take the input from the user in the main function.

Page 1

Page 2

Page 3

| Page | Visible Content                                                             | Extracted Requirements                                                                                                                                                                                                        | Status                          |
|------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| 1    | Lab title: <b>Lab 3 CSE225L - Classes &amp; Objects.</b><br>Objective list. | Implement and test <code>dynArr</code> using header/source files; demonstrate constructors/destructors; use <code>setValue</code> and <code>getValue</code> ; add <code>allocate(int s)</code> ; extend to 2D same-size rows. | Fully covered                   |
| 2    | Starter code for <code>dynarr.h</code> and <code>dynarr.cpp</code> .        | Class has private <code>int *data</code> , <code>int size</code> , default constructor, parameterized constructor, destructor, setter and getter.                                                                             | Cleaned, explained, implemented |
| 3    | Three tasks.                                                                | Task 1 driver file with two objects and five values; Task 2 add leak-free <code>allocate(int s)</code> ; Task 3 modify to support 2D array with rows and columns supplied by user.                                            | Implemented, traced, tested     |

**Starter-code weakness:** the original `getValue` and `setValue` do not check bounds. That matches beginner lab style, but during viva you should mention that invalid indexes can cause undefined behavior.

## SECTION 3

# Implementation Toolkit

Before code, build the mental machinery. This is the part that turns the lab from copy-paste into control.

# Section 3 · Implementation Toolkit

## 3.1 What This Lab Is Really About

This lab is not only about writing a class named `dynArr`. It is about teaching an object to own memory responsibly. The class becomes a small locked workshop: the raw pointer stays private, and the public methods are the only doors.

- **Private data** protects the pointer from random outside edits.
- **Constructors** decide how the object begins its life.
- **Destructor** cleans heap memory when the object leaves scope.
- **setValue/getValue** control how array elements are written and read.
- **allocate** changes memory size without abandoning the previous heap block.
- **2D extension** teaches that a matrix is usually a pointer to rows, and each row points to its own array.

## 3.2 Concepts Before Code

| Concept        | What It Means in This Lab                                        | Watch Out                                                         |
|----------------|------------------------------------------------------------------|-------------------------------------------------------------------|
| Class          | A blueprint for <code>dynArr</code> objects.                     | Put declaration in the header, definitions in source.             |
| Object         | A real instance such as <code>dynArr numbers(5);</code>          | Each object has its own <code>data</code> and <code>size</code> . |
| Constructor    | Runs automatically when the object is created.                   | Default constructor should set pointer to <code>NULL</code> .     |
| Destructor     | Runs automatically when the object is destroyed.                 | Use <code>delete[]</code> for arrays.                             |
| Dynamic memory | Memory created using <code>new</code> on the heap.               | Every <code>new[]</code> needs a matching <code>delete[]</code> . |
| 2D array       | An array of row pointers, where each row is a separate 1D array. | Delete every row first, then the row pointer array.               |

## 3.3 How to Recognize the Coding Pattern

| Task Pattern        | C++ Tool Needed                                             | First Move                                | Memory Concern                  | Common Trap                                                |
|---------------------|-------------------------------------------------------------|-------------------------------------------|---------------------------------|------------------------------------------------------------|
| Simple class        | <code>class</code> , fields, methods                        | Identify private data and public behavior | Object lifetime                 | Making the pointer public                                  |
| Header/source split | <code>.h</code> , <code>.cpp</code> , <code>main.cpp</code> | Put declarations in header                | All files must compile together | Defining methods twice or forgetting <code>#include</code> |

|                        |                                               |                                    |                                           |                                                            |
|------------------------|-----------------------------------------------|------------------------------------|-------------------------------------------|------------------------------------------------------------|
| Dynamic 1D array       | <code>new[]</code> ,<br><code>delete[]</code> | Read size or use constructor size  | Delete with <code>delete[]</code>         | Using <code>delete</code> instead of <code>delete[]</code> |
| Constructor/destructor | Special member functions                      | Initialize pointer and size        | Destructor releases memory                | Deleting uninitialized pointer                             |
| Resize method          | <code>allocate(int s)</code>                  | Allocate new block and copy values | Delete old block before replacing pointer | Losing old pointer before deletion                         |
| 2D equal-row array     | <code>int**</code> and nested loops           | Allocate row pointer array first   | Delete rows, then outer pointer           | Deleting only <code>data</code> , not each row             |

### 3.4 Lab Decision Tree

1. **Am I writing a class?** Put data members in `private`; put methods in `public`.
2. **Does the class use heap memory?** Add constructor and destructor. Initialize the pointer before using it.
3. **Is the task only testing the class?** Write the object creation and method calls in `main.cpp`.
4. **Do I need to resize?** Never overwrite the old pointer immediately. Create a new pointer, copy data, delete the old memory, then assign.
5. **Do I need 2D memory?** Allocate row pointers first, allocate each row second, use nested loops, delete in reverse order.

### 3.5 Required C++ Prerequisite Bridge

| Syntax                        | Meaning                                                        | Mini Example                                  |
|-------------------------------|----------------------------------------------------------------|-----------------------------------------------|
| <code>int *data;</code>       | Pointer that can store address of an integer or integer array. | <code>data = new int[5];</code>               |
| <code>data[index]</code>      | Accesses element at index in the dynamic array.                | <code>data[2] = 99;</code>                    |
| <code>delete[] data;</code>   | Frees an array created with <code>new[]</code> .               | Used in destructor.                           |
| <code>dynArr::setValue</code> | Defines a member function outside the class.                   | Needs <code>dynArr::</code> scope resolution. |
| <code>#ifndef</code>          | Header guard prevents duplicate inclusion.                     | Used in <code>dynarr.h</code> .               |

## Section 4 · Concept and Implementation Foundation

### 4.1 Required File Structure

#### **dynarr.h**

Contains the class declaration: private data members and public method prototypes. This file tells other files what the class can do.

#### **dynarr.cpp**

Contains method definitions. This is where constructors allocate memory, destructor deletes memory, and setters/getters access the array.

#### **main.cpp**

Contains the driver/test code. It creates objects, takes input, calls methods, and prints output. The lab specifically asks Task 1 to be done in this file.

### 4.2 Stack vs Heap Memory

Diagram · 1D dynArr object memory



Rule: `delete[] data;` when the object dies.

**Simple rule:** the pointer variable belongs to the object. The actual array created by `new int[s]` lives on the heap. The destructor must release that heap memory.

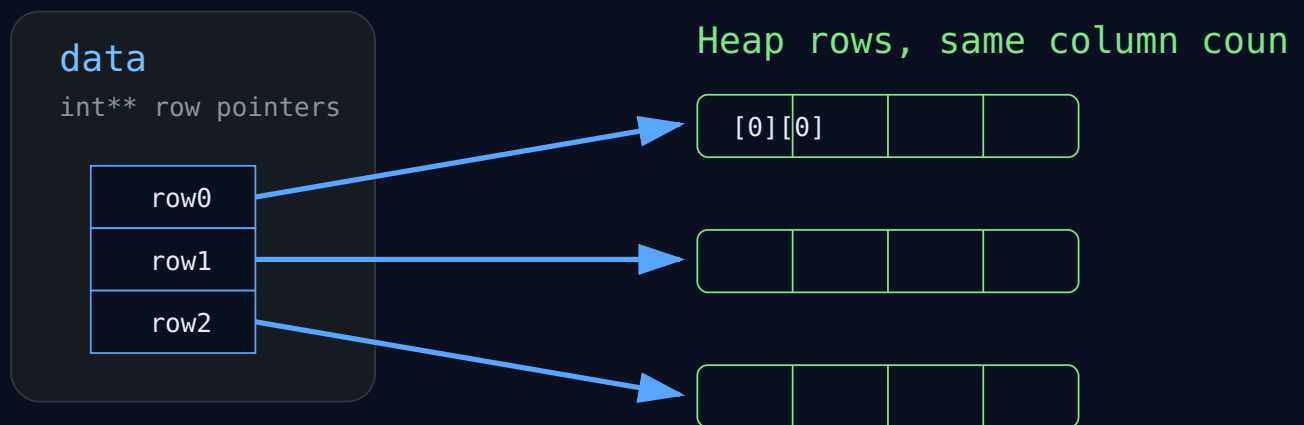
### 4.3 Starter Code Analysis

| Part                  | What It Already Does                                      | Limitation                                  |
|-----------------------|-----------------------------------------------------------|---------------------------------------------|
| <code>dynArr()</code> | Sets <code>data = NULL</code> and <code>size = 0</code> . | Object has no usable array until allocated. |

|                            |                                                          |                                             |
|----------------------------|----------------------------------------------------------|---------------------------------------------|
| <code>dynArr(int s)</code> | Allocates <code>s</code> integers on heap.               | No validation if <code>s</code> is invalid. |
| <code>~dynArr()</code>     | Deletes dynamic array using <code>delete[] data</code> . | Correct for 1D array.                       |
| <code>setValue</code>      | Writes value into <code>data[index]</code> .             | No index checking.                          |
| <code>getValue</code>      | Returns <code>data[index]</code> .                       | No index checking.                          |

## 4.4 2D Equal-Row Memory Model

Diagram · `int**` matrix layout



Delete order: `delete[]` each row first, then `delete[] data`.



## Section 5 · Algorithm Workflows

### Algorithm: Create 1D dynArr Object

Input: size *s*

Output: dynArr object with heap array

1. Store *s* in *size*.
2. Allocate data using `new int[s]`.
3. Use `setValue(index, value)` to write.
4. Use `getValue(index)` to read.
5. When object goes out of scope, destructor calls `delete[]` data.

### Algorithm: `allocate(int s)` for 1D Resize

Input: new size *s*

Output: object uses a new heap array of size *s*

1. If  $s \leq 0$ , delete old data and reset pointer to NULL.
2. Create a new `int` array of size *s*.
3. Copy `min(old size, new size)` values into the new array.
4. Fill new extra positions with 0.
5. Delete the old heap array using `delete[]`.
6. Point data to the new array.
7. Update *size*.

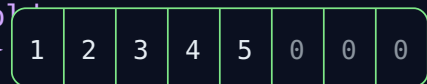
### Diagram · Resize without leak

Before `allocate(8)`



copy, delete old

After `allocate(8)`



#### The leak-proof move:

1. allocate `newData`
2. copy safe amount
3. `delete[]` old data
4. make data point to `newData`
5. update *size*

### Algorithm: Allocate 2D Equal-Row Array

Input: rows *r*, columns *c*

Output: matrix with *r* rows and *c* columns

1. Clear old matrix if one exists.
2. Store *r* in *rows* and *c* in *cols*.
3. Allocate `data = new int*[rows]`.
4. For every row *i*, allocate `data[i] = new int[cols]`.
5. Use nested loops to input values.
6. Use nested loops to print values.
7. In destructor, delete each `data[i]`, then `delete[]` data.

## SECTION 6

# Complete Lab Task Solutions

Three tasks, full code, sample runs, dry runs, bugs, complexity, viva sparks. The whole lab engine room.

## Section 6.1 · Task 1: Test the Original dynArr

### Task Statement

In `main.cpp`, create two objects of the class: one with no constructor argument and one with argument `5`. Take five input values from the user, store them inside the second object using `setValue`, then print all stored values using `getValue`.

### What You Need to Know First

- The default object proves that the no-argument constructor works.
- The second object, `dynArr numbers(5);`, owns a heap array of 5 integers.
- Only the methods can access the private pointer.
- Indexes run from `0` to `4`.

#### Pseudocode: Task 1 Driver

```
Create object emptyArray using default constructor.
Create object numbers using constructor argument 5.
Repeat i from 0 to 4:
    Read value.
    numbers.setValue(i, value).
Print "Stored values:".
Repeat i from 0 to 4:
    Print numbers.getValue(i).
```

● ● ● dynarr.h · Task 1

```
#ifndef DYNARR_H
#define DYNARR_H

class dynArr
{
private:
    int *data;
    int size;

public:
    dynArr();
    dynArr(int s);
    ~dynArr();
    void setValue(int index, int value);
    int getValue(int index);
};

#endif // DYNARR_H
```

● ● ● dynarr.cpp · Task 1

```
#include "dynarr.h"
#include <iostream>
using namespace std;

dynArr::dynArr()
{
    data = NULL;
    size = 0;
}

dynArr::dynArr(int s)
{
    data = new int[s];
    size = s;
}

dynArr::~dynArr()
{
    delete[] data;
}

int dynArr::getValue(int index)
{
    return data[index];
}

void dynArr::setValue(int index, int value)
{
    data[index] = value;
}
```

● ● ● main.cpp · Task 1

```
#include <iostream>
#include "dynarr.h"
using namespace std;

int main()
{
    dynArr emptyArray;
    dynArr numbers(5);

    int value;

    cout << "Enter 5 values: ";
    for (int i = 0; i < 5; i++)
    {
        cin >> value;
        numbers.setValue(i, value);
    }

    cout << "Stored values: ";
    for (int i = 0; i < 5; i++)
    {
        cout << numbers.getValue(i) << " ";
    }
    cout << endl;

    return 0;
}
```

### Sample Run · Task 1

Enter 5 values: 10 20 30 40 50  
Stored values: 10 20 30 40 50

## Code Explanation

| Code Part                                | Purpose                                                               | What to Watch                               |
|------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------|
| <code>dynArr emptyArray;</code>          | Calls default constructor.                                            | No heap array is allocated for this object. |
| <code>dynArr numbers(5);</code>          | Calls parameterized constructor and creates heap array of 5 integers. | Valid indexes are 0 to 4.                   |
| <code>numbers.setValue(i, value);</code> | Stores each user value in the private array.                          | Do not use index 5.                         |
| <code>numbers.getValue(i)</code>         | Reads values back from the array.                                     | Returns whatever is stored at that index.   |

## Dry Run

| Step | Variable/Object State                                         | Action                                | Output                           |
|------|---------------------------------------------------------------|---------------------------------------|----------------------------------|
| 1    | <code>emptyArray.data = NULL,</code><br><code>size = 0</code> | Create default object                 | -                                |
| 2    | <code>numbers.data -&gt; heap[5]</code>                       | Create object with size 5             | -                                |
| 3    | values: 10, 20, 30, 40, 50                                    | Call <code>setValue</code> five times | -                                |
| 4    | array contains 10 20 30 40 50                                 | Call <code>getValue</code> five times | Stored values: 10 20 30 40<br>50 |

**Common bugs:** using indexes 1 to 5 instead of 0 to 4, forgetting to include `dynarr.h` in `main.cpp`, compiling only `main.cpp` and not linking `dynarr.cpp`.

| Case                       | Expected Behavior                                                |
|----------------------------|------------------------------------------------------------------|
| Input five normal integers | Program prints the same five integers.                           |
| Input negative integers    | They are stored normally because array stores <code>int</code> . |
| Access index 5             | Unsafe. This is outside the 5-element array.                     |

**Viva sparks:** A constructor initializes an object. A destructor cleans resources. `data` is private so the array cannot be changed directly from outside. `delete[]` is required because the memory was created with `new[]`.

## Section 6.2 · Task 2: Add allocate(int s)

### Task Statement

Modify the header and source files to add `void allocate(int s)`, allowing the size of the dynamic array to change. Ensure that memory is not leaked.

### What You Need to Know First

The danger in resizing is pointer amnesia. If you write `data = new int[s];` while `data` already points to old memory, the old address is lost forever. That is a memory leak. The safe version stores the new block separately, copies values, deletes the old block, then redirects the pointer.

Pseudocode: Task 2 `allocate(int s)`

```
If new size is invalid:
    delete old data
    set data to NULL
    set size to 0
    stop
Create newData with new int[s]
Copy old values up to the smaller size
Fill new empty cells with 0
Delete old data using delete[]
Set data = newData
Set size = s
```

● ● ● dynarr.h · Task 2

```
#ifndef DYNARR_H
#define DYNARR_H

class dynArr
{
private:
    int *data;
    int size;

public:
    dynArr();
    dynArr(int s);
    ~dynArr();
    void allocate(int s);
    void setValue(int index, int value);
    int getValue(int index);
    int getSize();
};

#endif // DYNARR_H
```

```
#include "dynarr.h"
#include <iostream>
using namespace std;

dynArr::dynArr()
{
    data = NULL;
    size = 0;
}

dynArr::dynArr(int s)
{
    data = new int[s];
    size = s;
}

dynArr::~dynArr()
{
    delete[] data;
}

void dynArr::allocate(int s)
{
    if (s <= 0)
    {
        delete[] data;
        data = NULL;
        size = 0;
        return;
    }

    int *newData = new int[s];
    int limit = (s < size) ? s : size;

    for (int i = 0; i < limit; i++)
    {
        newData[i] = data[i];
    }

    for (int i = limit; i < s; i++)
    {
        newData[i] = 0;
    }
}
```

```
    }

    delete[] data;
    data = newData;
    size = s;
}

int dynArr::getValue(int index)
{
    return data[index];
}

void dynArr::setValue(int index, int value)
{
    data[index] = value;
}

int dynArr::getSize()
{
    return size;
}
```

```
#include <iostream>
#include "dynarr.h"
using namespace std;

int main()
{
    dynArr numbers(5);

    cout << "Enter 5 values: ";
    for (int i = 0; i < 5; i++)
    {
        int value;
        cin >> value;
        numbers.setValue(i, value);
    }

    numbers.allocate(8);

    cout << "After resizing to 8, enter 3 more values: ";
    for (int i = 5; i < 8; i++)
    {
        int value;
        cin >> value;
        numbers.setValue(i, value);
    }

    cout << "Array values: ";
    for (int i = 0; i < numbers.getSize(); i++)
    {
        cout << numbers.getValue(i) << " ";
    }
    cout << endl;

    return 0;
}
```



## Sample Run · Task 2

Enter 5 values: 1 2 3 4 5  
After resizing to 8, enter 3 more values: 6 7 8  
Array values: 1 2 3 4 5 6 7 8

## Code Explanation

| Code Part                                      | Purpose                                          | What to Watch                                   |
|------------------------------------------------|--------------------------------------------------|-------------------------------------------------|
| <code>int *newData = new int[s];</code>        | Creates the new heap block.                      | Do not overwrite <code>data</code> yet.         |
| <code>limit = (s &lt; size) ? s : size;</code> | Determines how many old values are safe to copy. | Prevents reading outside old array.             |
| <code>delete[] data;</code>                    | Frees old heap block.                            | This must happen before losing the old address. |
| <code>data = newData;</code>                   | Makes object point to new array.                 | After this, the object owns new memory.         |

## Dry Run

| Moment           | Pointer                     | Heap Content             | Status          |
|------------------|-----------------------------|--------------------------|-----------------|
| Before resize    | <code>data</code>           | [1, 2, 3, 4, 5]          | Old block alive |
| Create new block | <code>newData</code>        | [?, ?, ?, ?, ?, ?, ?, ?] | New block alive |
| Copy and fill    | <code>newData</code>        | [1, 2, 3, 4, 5, 0, 0, 0] | Safe copy       |
| Delete old       | <code>data</code>           | old block released       | No leak         |
| Redirect         | <code>data = newData</code> | [1, 2, 3, 4, 5, 0, 0, 0] | Resize complete |

**Instructor-note version:** some beginner labs accept a shorter `allocate` that deletes the old array and creates a fresh one without preserving values. The guide version keeps old values because it is safer and demonstrates real resizing.

**Common bugs:** assigning `data = new int[s]` before deleting old memory, using `delete data` instead of `delete[] data`, not updating `size`, reading old indexes after shrinking.

| Operation                    | Time Complexity                                      | Space Complexity |
|------------------------------|------------------------------------------------------|------------------|
| <code>setValue</code>        | O(1)                                                 | O(1)             |
| <code>getValue</code>        | O(1)                                                 | O(1)             |
| <code>allocate(int s)</code> | O(min(old size, new size) + new size initialization) | O(new size)      |

**Viva sparks:** Memory leak means allocated memory is no longer reachable. `allocate` must delete old memory before replacing the pointer. Shrinking may lose extra values. Growing needs initialization for new cells if you want predictable output.

## Section 6.3 · Task 3: Extend dynArr to 2D

### Task Statement

Modify the header and source files again to support a two-dimensional array where all rows have the same column size. The user specifies rows, columns, and the matrix content in `main.cpp`.

### What You Need to Know First

- A 2D dynamic array with equal columns can be represented using `int **data`.
- `data` points to an array of row pointers.
- Each row pointer points to an array of integers.
- Input and output require nested loops: outer loop for rows, inner loop for columns.
- Deletion must be nested too: delete each row, then delete the row pointer array.

#### Pseudocode: Task 3 2D dynArr

```
Read rows and cols in main.
Create dynArr matrix(rows, cols).
For i from 0 to rows - 1:
    For j from 0 to cols - 1:
        Read value.
        matrix.setValue(i, j, value).
For i from 0 to rows - 1:
    For j from 0 to cols - 1:
        Print matrix.getValue(i, j).
When matrix goes out of scope:
    Destructor deletes every row.
    Destructor deletes the row-pointer array.
```

```
#ifndef DYNARR_H
#define DYNARR_H

class dynArr
{
private:
    int **data;
    int rows;
    int cols;

    void clear();

public:
    dynArr();
    dynArr(int r, int c);
    ~dynArr();

    void allocate(int r, int c);
    void setValue(int row, int col, int value);
    int getValue(int row, int col);
    int getRows();
    int getCols();
};

#endif // DYNARR_H
```

```
#include "dynarr.h"
#include <iostream>
using namespace std;

dynArr::dynArr()
{
    data = NULL;
    rows = 0;
    cols = 0;
}

dynArr::dynArr(int r, int c)
{
    data = NULL;
    rows = 0;
    cols = 0;
    allocate(r, c);
}

dynArr::~dynArr()
{
    clear();
}

void dynArr::clear()
{
    if (data != NULL)
    {
        for (int i = 0; i < rows; i++)
        {
            delete[] data[i];
        }
        delete[] data;
    }

    data = NULL;
    rows = 0;
    cols = 0;
}

void dynArr::allocate(int r, int c)
{

```

```
clear();

if (r <= 0 || c <= 0)
{
    return;
}

rows = r;
cols = c;
data = new int*[rows];

for (int i = 0; i < rows; i++)
{
    data[i] = new int[cols];
    for (int j = 0; j < cols; j++)
    {
        data[i][j] = 0;
    }
}

void dynArr::setValue(int row, int col, int value)
{
    data[row][col] = value;
}

int dynArr::getValue(int row, int col)
{
    return data[row][col];
}

int dynArr::getRows()
{
    return rows;
}

int dynArr::getCols()
{
    return cols;
}
```

```

#include <iostream>
#include "dynarr.h"
using namespace std;

int main()
{
    int rows, cols;

    cout << "Enter number of rows: ";
    cin >> rows;

    cout << "Enter number of columns: ";
    cin >> cols;

    dynArr matrix(rows, cols);

    cout << "Enter " << rows * cols << " values:" << endl;
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            int value;
            cin >> value;
            matrix.setValue(i, j, value);
        }
    }

    cout << "Matrix values:" << endl;
    for (int i = 0; i < matrix.getRows(); i++)
    {
        for (int j = 0; j < matrix.getCols(); j++)
        {
            cout << matrix.getValue(i, j) << " ";
        }
        cout << endl;
    }

    return 0;
}

```

## Sample Run · Task 3

```

Enter number of rows: 2
Enter number of columns: 3
Enter 6 values:
1 2 3 4 5 6
Matrix values:
1 2 3
4 5 6

```

## Code Explanation

| Code Part                           | Purpose                                  | What to Watch                                  |
|-------------------------------------|------------------------------------------|------------------------------------------------|
| <code>int **data;</code>            | Stores address of the row-pointer array. | It is not the same as <code>int *data</code> . |
| <code>data = new int*[rows];</code> | Creates array of row pointers.           | Rows themselves are not allocated yet.         |
|                                     | Creates each row.                        | Must repeat for every row.                     |

|                                       |                                                  |                                                 |
|---------------------------------------|--------------------------------------------------|-------------------------------------------------|
| <code>data[i] = new int[cols];</code> |                                                  |                                                 |
| <code>clear()</code>                  | Deletes old matrix safely and resets the object. | Prevents duplicate cleanup logic.               |
| <code>delete[] data[i]</code>         | Deletes one row.                                 | Must happen before <code>delete[] data</code> . |

## Dry Run: 2 rows, 3 columns

| Row/Column | Input Value | Stored At               |
|------------|-------------|-------------------------|
| i=0, j=0   | 1           | <code>data[0][0]</code> |
| i=0, j=1   | 2           | <code>data[0][1]</code> |
| i=0, j=2   | 3           | <code>data[0][2]</code> |
| i=1, j=0   | 4           | <code>data[1][0]</code> |
| i=1, j=1   | 5           | <code>data[1][1]</code> |
| i=1, j=2   | 6           | <code>data[1][2]</code> |

**Common bugs:** allocating only `data = new int*[rows]` and forgetting each row, using one loop instead of nested loops, deleting `data` before deleting rows, using `row` and `col` in reverse order.

| Operation       | Time Complexity                     | Space Complexity                    |
|-----------------|-------------------------------------|-------------------------------------|
| Allocate matrix | $O(\text{rows} \times \text{cols})$ | $O(\text{rows} \times \text{cols})$ |
| Set one value   | $O(1)$                              | $O(1)$                              |
| Get one value   | $O(1)$                              | $O(1)$                              |
| Print matrix    | $O(\text{rows} \times \text{cols})$ | $O(1)$                              |
| Delete matrix   | $O(\text{rows})$                    | $O(1)$                              |

**Viva sparks:** A 2D dynamic array built with `int**` is an array of row pointers. You must allocate every row separately. Deletion order matters because once the outer pointer is deleted, the row addresses are lost.



## Section 7 · Debugging and Common Errors

| Symptom                                                       | Likely Cause                                                          | Fix                                                                                       |
|---------------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Linker error: undefined reference to <code>dynArr::...</code> | <code>dynarr.cpp</code> was not compiled with <code>main.cpp</code> . | Build all files together.                                                                 |
| Program crashes when setting value                            | Index is outside array bounds or object has no allocated array.       | Use valid indexes and create object with size.                                            |
| Garbage values printed                                        | Reading uninitialized cells.                                          | Set every value before printing or initialize new cells.                                  |
| Memory leak after resize                                      | Old pointer was overwritten before deletion.                          | Use temporary pointer and <code>delete[]</code> old data.                                 |
| 2D destructor crashes                                         | Deleting rows after outer pointer was already deleted.                | Delete each row first, then outer pointer.                                                |
| Header included multiple times                                | No include guard.                                                     | Use <code>#ifndef DYNARR_H</code> , <code>#define DYNARR_H</code> , <code>#endif</code> . |

**Danger zone:** raw pointers are sharp tools. The code can compile and still be wrong if memory ownership is mishandled. Compilation only checks syntax; tracing checks survival.

### Build Command

```
g++ main.cpp dynarr.cpp -o lab3
./lab3
```

## Section 8 · Testing Checklist

|   | Test                              | Why                                                                       |
|---|-----------------------------------|---------------------------------------------------------------------------|
| ✓ | Create default object.            | Confirms default constructor works and destructor can delete NULL safely. |
| ✓ | Create 1D object with size 5.     | Matches Task 1 exactly.                                                   |
| ✓ | Input five positive values.       | Basic expected case.                                                      |
| ✓ | Input negative and zero values.   | Confirms array stores general integers.                                   |
| ✓ | Resize from 5 to 8.               | Checks growing and old-value preservation.                                |
| ✓ | Resize from 5 to 3.               | Checks shrinking behavior.                                                |
| ✓ | Allocate 2D matrix $2 \times 3$ . | Checks standard nested allocation.                                        |
| ✓ | Allocate 2D matrix $1 \times 1$ . | Checks smallest valid matrix.                                             |
| ✓ | Build in multi-file project.      | Confirms header/source split is correct.                                  |

## | Section 9 · Complexity Summary

### 1D dynArr Operations

| Operation             | Time                  | Reason                                                                   | Space  |
|-----------------------|-----------------------|--------------------------------------------------------------------------|--------|
| Constructor with size | $O(n)$                | Allocation reserves $n$ integers; if initialized, $n$ cells are touched. | $O(n)$ |
| <code>setValue</code> | $O(1)$                | Direct index access.                                                     | $O(1)$ |
| <code>getValue</code> | $O(1)$                | Direct index access.                                                     | $O(1)$ |
| <code>allocate</code> | $O(n)$                | Copies old values and initializes new cells.                             | $O(n)$ |
| Destructor            | $O(1)$ from code view | One <code>delete[]</code> call; runtime frees the block.                 | $O(1)$ |

### 2D dynArr Operations

| Operation                      | Time            | Reason                                  | Space           |
|--------------------------------|-----------------|-----------------------------------------|-----------------|
| Allocate rows $\times$ columns | $O(r \times c)$ | Creates rows and initializes each cell. | $O(r \times c)$ |
| Set one cell                   | $O(1)$          | Direct row-column access.               | $O(1)$          |
| Get one cell                   | $O(1)$          | Direct row-column access.               | $O(1)$          |
| Print all cells                | $O(r \times c)$ | Nested loops visit every cell once.     | $O(1)$          |
| Destructor                     | $O(r)$          | Deletes each row pointer once.          | $O(1)$          |

## Section 10 · Quick Revision Sheet

### Syntax Patterns

- `data = new int[s];`
- `delete[] data;`
- `data = new int*[rows];`
- `data[i] = new int[cols];`
- `delete[] data[i];`
- `ClassName::functionName`

### Pointer Safety Rules

- Initialize pointers to `NULL`.
- Do not overwrite a live pointer before deleting its memory.
- Use `delete[]` for arrays.
- After deleting, set pointer to `NULL` if the object may continue living.
- Never use indexes outside valid range.

### Common Viva Questions

| Question                                                           | Short Answer                                                                                                         |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Why is <code>data</code> private?                                  | To protect memory from direct outside access and force controlled access through methods.                            |
| What is the purpose of the destructor?                             | It releases heap memory when the object is destroyed.                                                                |
| Difference between <code>delete</code> and <code>delete[]</code> ? | <code>delete</code> frees one object; <code>delete[]</code> frees an array.                                          |
| Why use header guards?                                             | To prevent the same class declaration from being included multiple times.                                            |
| How do you delete a 2D dynamic array?                              | Delete each row with <code>delete[] data[i]</code> , then delete the outer pointer with <code>delete[] data</code> . |
| What is a memory leak?                                             | Allocated heap memory becomes unreachable and cannot be freed.                                                       |

**Submission checklist:** include `dynarr.h`, `dynarr.cpp`, and `main.cpp`; compile all files together; test every task; be ready to explain constructor, destructor, `setValue`, `getValue`, and deletion order.

## Section 11 · Lab Coverage Audit

| Page/<br>Slide<br>Range | Task or Topic Found                                                                                  | Covered In             | Status                            | Notes                                           |
|-------------------------|------------------------------------------------------------------------------------------------------|------------------------|-----------------------------------|-------------------------------------------------|
| Page 1                  | Lab 3 title and objectives: classes, objects, constructors/destructors, set/get, allocate, 2D array. | Sections 1, 3, 4, 5, 6 | Explained                         | No unreadable content.                          |
| Page 2                  | Starter <code>dynarr.h</code> and <code>dynarr.cpp</code> .                                          | Sections 4 and 6.1     | Implemented, corrected formatting | Bounds-check weakness noted.                    |
| Page 3 · Task 1         | Create two objects, input five values, store and print values.                                       | Section 6.1            | Implemented, traced, tested       | Driver file provided.                           |
| Page 3 · Task 2         | Add <code>allocate(int s)</code> and avoid memory leak.                                              | Section 6.2            | Implemented, traced, tested       | Preserves old values and initializes new cells. |
| Page 3 · Task 3         | Modify class for 2D array with equal row size; input rows, columns, and contents from main.          | Section 6.3            | Implemented, diagrammed, tested   | Deletion order explained with memory diagram.   |

### Final Validation

The code versions in this guide were compiled with `g++ -std=c++11`. The sample terminal outputs were produced from the compiled programs. Layout was rendered and checked after PDF creation.